Hosted Demo - openai key
Local Demo - hosting model your model locally
Openshift AI demo - MaaS endpoint
Slide 33 handover after demo
Slide 54 maarten

Presenter checkpoints (45 minutes, including demos)

- **6:00 Slide 13 Coffee setup done, vocabulary begins**
- **12:00 Slide 20 Hosted demo complete**
- **18:00 Slide 28 Data boundaries and personal scenarios**
- **21:00 Slide 33 Local demo complete**
- **30:00 Slide 41 Serving stack complete**
- **38:00 Slide 48 MaaS demo complete, compare all three**

42:00 Slide 54 Red Hat and closing sequence

45:00 Slide 57 Thank you

DEVOXX

Owning the inference layer

When and how to run your own models



Camille Nigon

Specialist Solutions Architect · Red Hat

linkedin.com/in/camille-nigon
https://github.com/camille-maarten/presentations

Maarten Vandeperre

Sr. Specialist Solutions Architect (AppDev & AI) · Red Hat
Advocate digital sovereignty · cloud-native development · platform engineering

maarten-vandeperre.be · linkedin.com/in/maarten-vandeperre
https://github.com/camille-maarten/presentations

Rate the talks. Please.

Devoxx asks for a rating at the end of every session. Here is the honest reason it matters.

Feedback makes it better

The comments tell me which analogy landed and which slide lost the room.

Ratings feed acceptance

Committees look at scores when deciding who gets to speak again.

Under 10 votes: ignored

Below the threshold the score doesn't count at all.

The trap: people who didn't enjoy a talk always vote.
People who did tend to walk out happy and forget. So if you like it, please say so: otherwise the only voices on record are the unhappy ones.

Scan to rate

A quick word about Red Hat

Red Hat builds on open source projects and supports them for production use.



RHEL

Red Hat Enterprise Linux

Linux for the containers, GPUs and drivers underneath.



OpenShift

The Kubernetes platform

Container orchestration, networking and tools for developers.



Ansible

Automation

Provisioning and configuration without repeating the same manual work.



OpenShift AI

The AI platform

Model serving, pipelines, model registry, guardrails and Model as a Service.

vLLM · llm-d · Kubernetes · Keycloak · Kuadrant · Podman · TrustyAI · NeMo guardrails

We contribute to these open source projects and use them in the stack we're discussing today.

What we bring: Open source tools · integration and support

developers.redhat.com free developer subscription, articles, tutorials

Red Hat

Let's start a company!



The idea is under NDA.

The infrastructure isn't.

We've hired four people



What do we need first?

Coffee, of course



No coffee.

No IT.

To Brew or not to Brew?

The coffee shop or our own machine?



Camille Nigon

Specialist Solutions Architect · Red Hat

linkedin.com/in/camille-nigon

Maarten Vandeperre

Sr. Specialist Solutions Architect (AppDev & AI) · Red Hat
Advocate digital sovereignty · cloud-native development · platform engineering

maarten-vandeperre.be · linkedin.com/in/maarten-vandeperre

To Brew or not to Brew?

Keep your options open



The coffee shop or our own machine?

Camille Nigon

Specialist Solutions Architect · Red Hat

linkedin.com/in/camille-nigon

Maarten Vandeperre

Sr. Specialist Solutions Architect (AppDev & AI) · Red Hat
Advocate digital sovereignty · cloud-native development · platform engineering

maarten-vandeperre.be · linkedin.com/in/maarten-vandeperre

What we'll cover

A coffee order in Java

Hosted API, local model, & shared endpoint

Validation, data boundaries & local model hosting

Decision framework



How do we get our coffee?



Hosted API

Managed platform

Our own serving stack

Platform engineering 101 (Devoxx Belgium): https://youtu.be/DaYNwrvJGmk?si=22ZzXD8rxgh4w8BD

Coffee, AI and Java



Training
Learning the recipe

Inference
Brewing a response

Tokens
Measuring the payload - the order

A model needs a runtime

The model: the recipe

Learned weights and a tokenizer
The files the runtime loads

The runtime: the machine

vLLM serves model requests
A role like JBoss or Tomcat

The model is an artefact. The runtime makes it a service.



Reference: coffee, AI and Java

AI term	At the coffee shop	For a Java developer
Inference	Brewing a cup	Execute a model request and read the response
Tokens	Coffee in, cups out	Input and output units, often used for billing
vLLM / runtime	Espresso machine	A serving role like JBoss or Tomcat
Quantisation	Adjusting the grind	Fewer bits per value. Retest quality and speed
KV / prefix cache	Cups already warmed	Reuse computed context and matching prefixes
Model as a Service	The cafeteria counter	An internal REST API run by the platform team
Portability	A different supplier	Change backends while keeping domain code



Option 1: Let the coffeehouse handle it

What Could Possibly Go Wrong? Look closely. How many issues can you find?



Option 1: Let the coffeehouse handle it

What Could Possibly Go Wrong? Look closely. How many issues can you find?



Challenge: The hosted API on day two

Pricing is theirs

Coffeehouse: the price on the board can change overnight.
Hosted API: pricing and unit economics can change without you controlling the decision.

Usage attribution can get messy

Coffeehouse: everybody uses the same company card.
Hosted API: one shared API key can hide which team, app or agent consumed the tokens.

You share the capacity

Coffeehouse: your wait depends on how busy the café is.
Hosted API: latency, throughput and rate limits are partly outside your control.

The recipe can change

Coffeehouse: same drink name, but beans or recipe changed.
Hosted API: the model name may stay while checkpoint or behaviour changes.

Your prompts leave your environment

Coffeehouse: your name, order and payment details leave with the transaction.
Hosted API: prompts may contain documents, code or customer data.

Their lifecycle becomes yours

Coffeehouse: it can close, change owner, or remove your favourite drink.
Hosted API: deprecations and regional restrictions become app dependencies.

You get the items they expose

Coffeehouse: you choose from the menu, not every brewing parameter.
Hosted API: low-level serving controls are usually hidden.

It remains operational expenditure

Coffeehouse: every cup is another expense.
Hosted API: every token is recurring cost; no infrastructure asset remains.

Bottom line: *a hosted API removes a lot of operational responsibility, but it also moves many important decisions to the provider. That is not necessarily bad — it is the trade-off you need to understand.*



Challenge: The hosted API on day two

Bills and rate limits

Usage grows. Prices and quotas can change.

Model behaviour changes

An update can change the same prompt's answer.

Data leaves our boundary

Check prompts, logs and retention terms.

Our Java code owns the order

Guardrailing: Validate JSON and handle failures before acting.



AI in a Java application

A model response is untrusted input.

Quarkus owns the order rules.
A replaceable client calls the model.

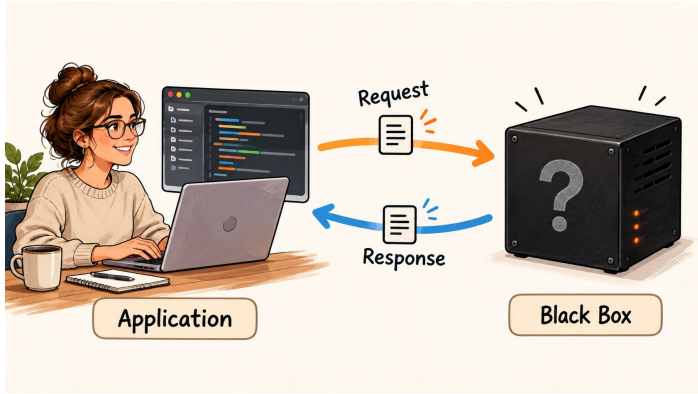
0% of the UI and Java validation code changes across all three demos.
Only the inference endpoint configuration changes.



Demo 01: the hosted barista

OpenAI

The model proposes. Quarkus validates.



Quarkus · Gradle · Java 25
React · Vite · Tailwind

GitHub: https://github.com/maarten-vandeperre/owning-the-inference-layer

The safety boundary lives in Quarkus

Model output

Untrusted JSON
A proposed order

→

Quarkus

Menu, size, quantity
Server-side price

→

Customer

Reviews the quote
Explicitly confirms

Only confirmation lets Quarkus write the order.



Two large oat lattes, please

“Two large oat lattes, please.”

Natural language

2 × large oat latte

€10.20

Review, then confirm

Quarkus calculates the price. Confirmation creates the order.



Why run our own models now?



Red Hat validated models: https://www.redhat.com/en/products/ai/validated-models

More models to choose from

Serving tools like vLLM

Results from real applications

Why choose self-hosting?

Customisation

Fine-tuning for our task or domain

Data restrictions

Customer contracts and regulatory requirements

Cost

The API bill is getting expensive

Latency

More control over response times

Sometimes one requirement makes the decision for us.

Multi-model vs foundation model: https://youtu.be/cBZHt356sRY?si=l7V8bkLOlxqHqaUb



What do we want to control?



Our data

Where it goes and who can access it

Continuity

What happens if the provider cuts access?

Model lifecycle

When we upgrade or retire a version

Portability

What it costs and whether we can switch

Portability is loose coupling



Business logic

Menu rules and pricing stay in Java

Inference client

A replaceable REST boundary

Configuration

Endpoint, model and credential

Acceptance tests

The same order cases for every provider

Same app. Different supplier.



Data residency isn’t sovereignty



Data residency

Where?

Storage and processing location

Sovereignty

Who has control?

Access, keys and jurisdiction (US Cloud ACT)
Updates, operations and portability

Keep your options open

An EU region tells us the location. We still need to check who has control.



Sovereignty: can the provider prove it?

Marketing sovereignty	Real sovereignty: evidence
“It runs in Europe.”	Data paths, access and jurisdiction
“It is open.” (but open source?)	Usable artefacts (ref. OKD), licences and alternatives
“You are in control.”	Keys, admin rights and recovery tests
“You can always leave.”	A rehearsed exit, with time and cost

We should ask every provider for proof, including Red Hat.



Where else does the data go?

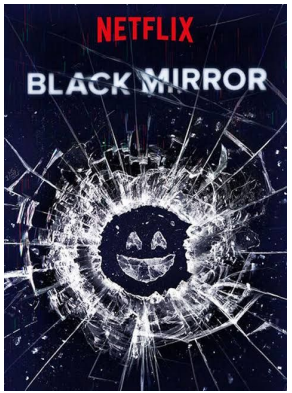


Prompts and responses

Logs and telemetry

Tools and fallback services

What if it becomes personal?



What if your doctor uses ChatGPT
and your medical details reach an insurer?
Could that affect your mortgage?

ABC News · 29 Sep 2026

HIPAA Privacy Rule (U.S.)

What if a company uses personal data
shared with an AI provider
to decide how much to pay you? (Ref. Uber)

The Guardian · 3 May 2026

Hypothetical scenarios. Linked reporting provides context.

Option 2: Brew it ourselves

What Could Possibly Go Wrong? More control. More choices. More complexity.



Option 2: Brew it ourselves

What Could Possibly Go Wrong? More control. More choices. More complexity.



Challenge: Owning inference on day two

Up-front cost

Coffee machine: you buy the machine first.
Inference: GPUs or reserved capacity become your investment before the first token is generated.

Idle capacity

Coffee machine: it sits unused for much of the day.
Inference: idle GPUs still cost money, even when nobody is prompting.

Peak demand

Coffee machine: everybody wants coffee at the same time.
Inference: traffic bursts, queues and GPU scaling become your problem.

Things keep evolving

Coffee machine: new beans, roasts and recipes appear.
Inference: new models, quantization methods and serving techniques keep moving.

Security and maintenance

Coffee machine: the lock, cleaning and maintenance are yours.
Inference: patching, CVEs, access control and model provenance are yours.

Failures are yours

Coffee machine: if it breaks, you need a backup plan.
Inference: you need monitoring, redundancy, runbooks and fallback paths.

You are limited by what you own

Coffee machine: your machine determines what drinks you can make.
Inference: your GPUs, memory and architecture determine which models you can run.

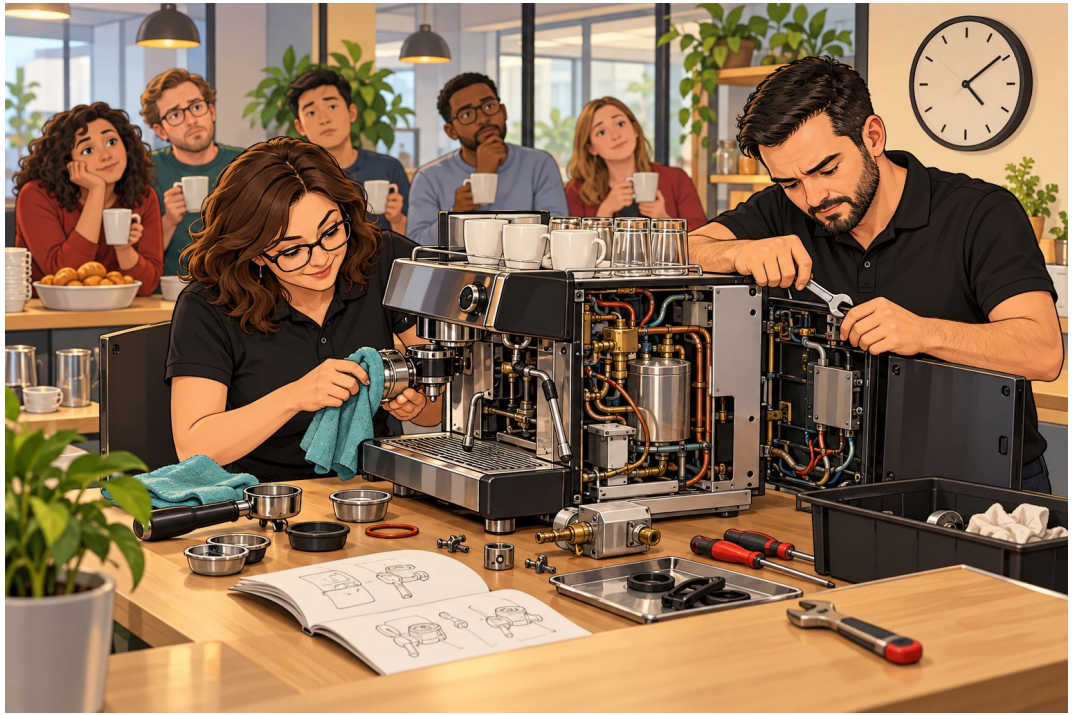
You need the skills

Coffee machine: somebody must learn grind size, pressure and tuning.
Inference: somebody must understand vLLM, scheduling, KV cache, quantization and performance tuning.

Bottom line: owning inference moves cost, responsibility and complexity to your side of the counter — but it also gives you control, visibility and choice.



Challenge: Who keeps the service running?

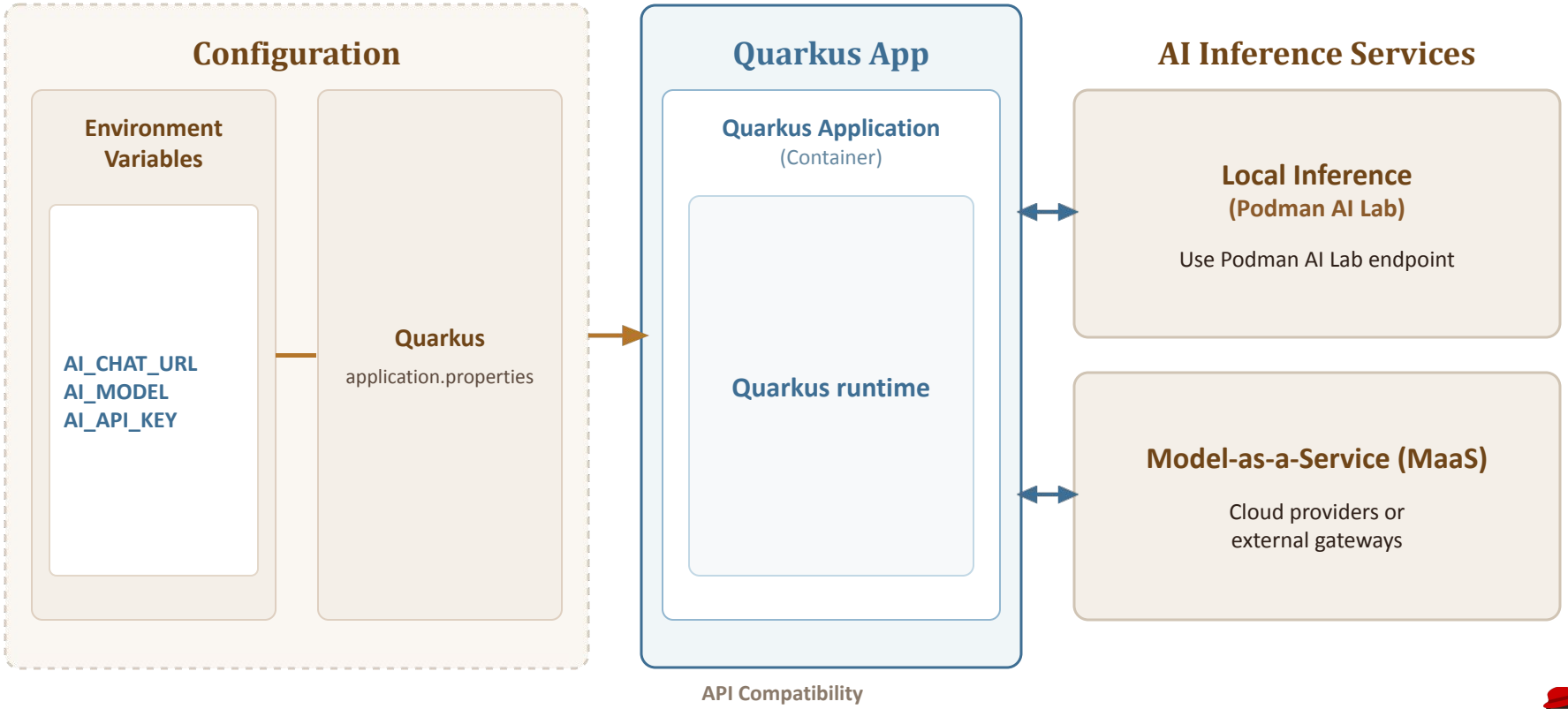


Enough capacity?

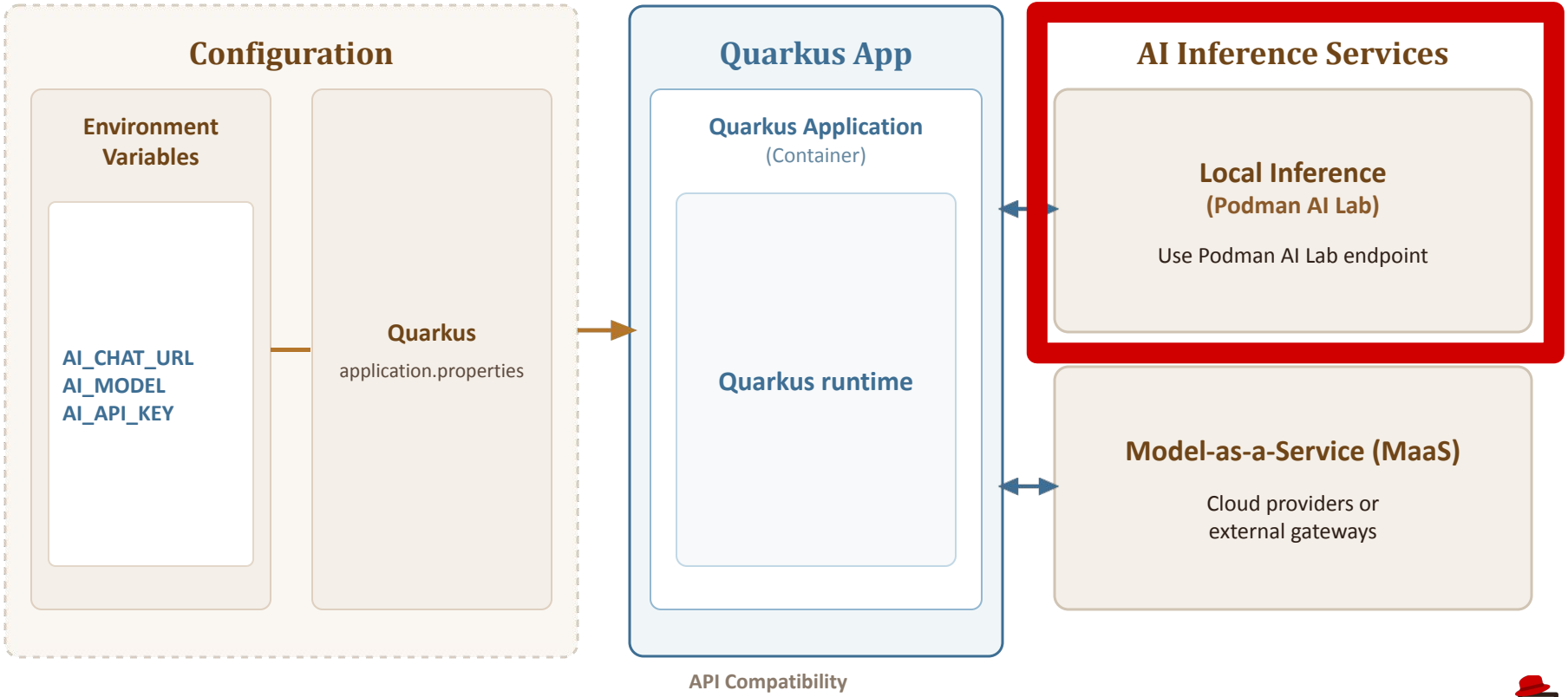
What if it fails?

Who's on call?

Quarkus to AI Inference Architecture



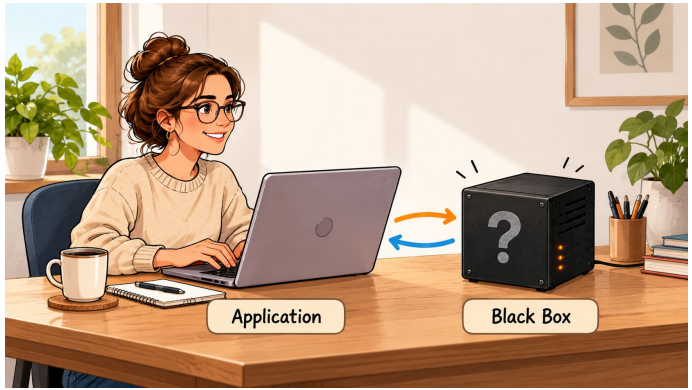
Quarkus to AI Inference Architecture



Demo 02: the local barista

Podman AI Lab

Local requests. Same Java checks.



Quarkus · Gradle · Java 25
React · Vite · Tailwind

GitHub: https://github.com/maarten-vandeperre/owning-the-inference-layer

Portability through configuration

AI_CHAT_URL	Hosted, local or shared endpoint
AI_MODEL	The model exposed by that service
AI_API_KEY	Service credential, kept in Quarkus

Same React UI. Same Java validation. Different configuration.



Challenges: What does the app need?



The right order

A short enough wait

Enough capacity at peak time (scale)

How long does the customer wait?



Until the first token

Between tokens

Until the order is ready

What does each completed order cost?



Completed orders

Capacity, including idle time

Running and supporting it

Quantisation: adjusting the grind

Quantisation

Fewer bits per value
Less model memory

The taste test

Does it still get the order right?
Is it fast enough?

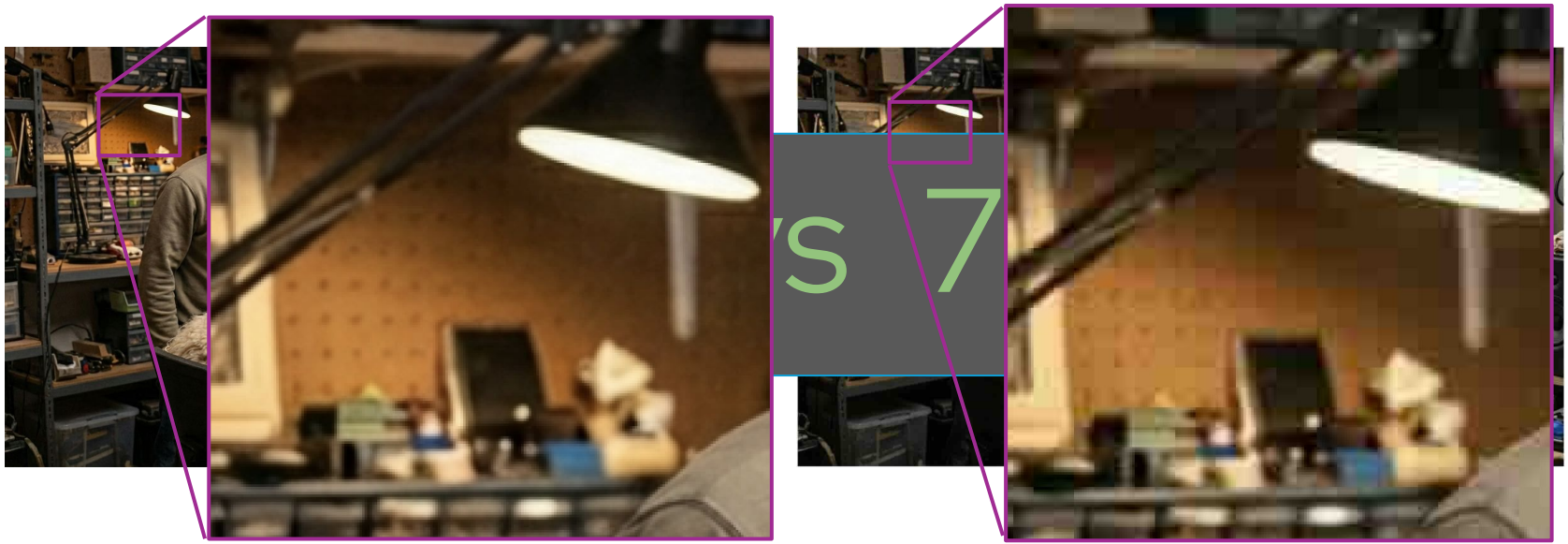
Less memory can help. Quality and speed still need testing.



Quantisation

uncompressed - original size

compressed - quantised



Can you see the difference?

vLLM: the commercial espresso machine



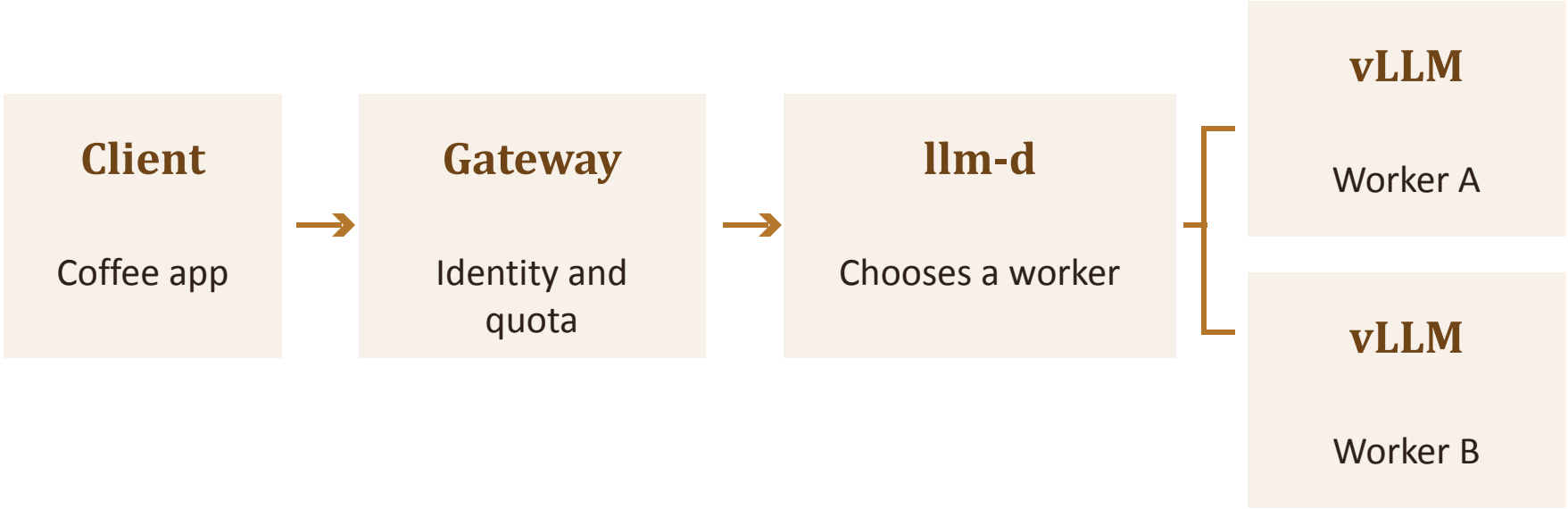
Serves model requests

KV cache
Reuses context

Batching
Shares capacity

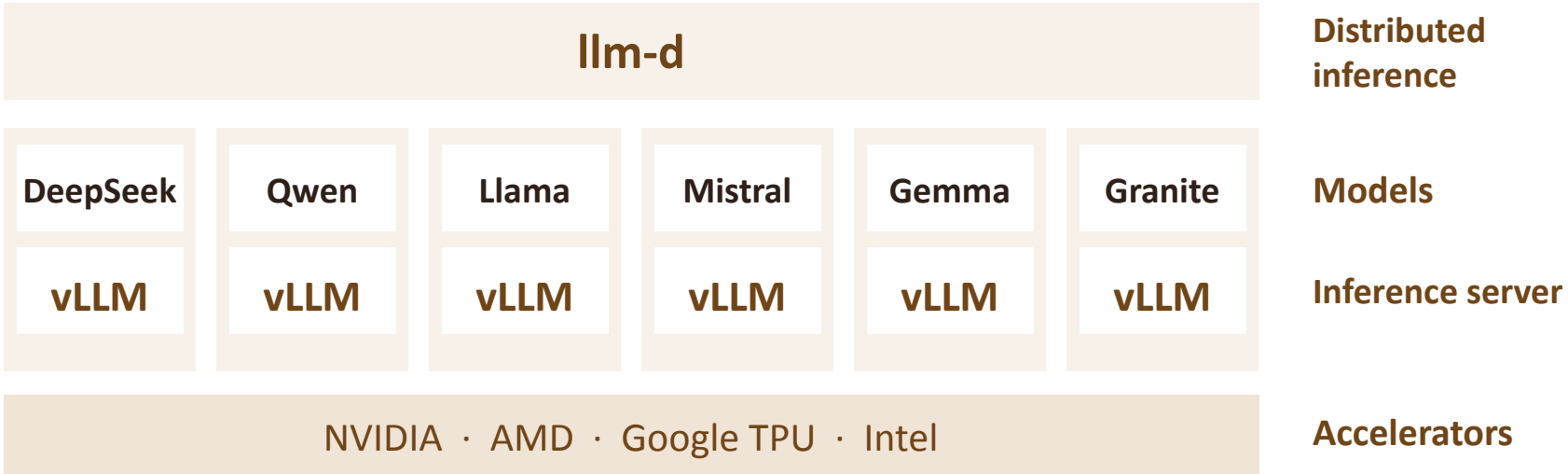
Which worker should get the request?

llm-d considers request length, waiting work and cached context.



What's in the serving stack?

Model, runtime and hardware are separate choices



The counter, floor manager, recipe, machine and power supply

Now every team wants a machine





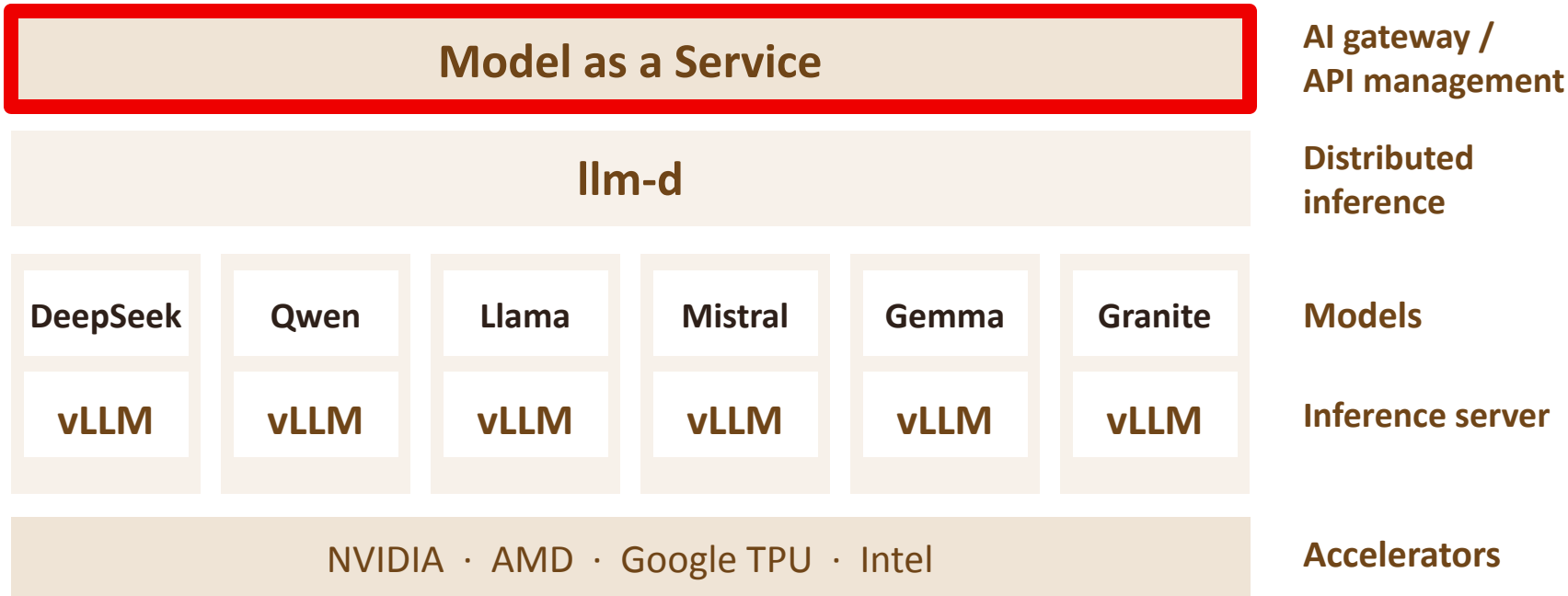
Everyone repeats the setup

GPUs sit in separate silos

Who is on call?

What's in the serving stack?

Model, runtime and hardware are separate choices



The counter, floor manager, recipe, machine and power supply

The gateway is our coffee counter



Who can use it?

How much can they use?

Where do requests go?

Platform engineering 101 (Devoxx Belgium): https://youtu.be/DaYNwrvJGmk?si=22ZzXD8rxgh4w8BD

MaaS / AI Gateway

Applications and agents

Web services, microservices, chatbots and autonomous intelligent agents

API key + OpenAI compatible API

MaaS / AI Gateway

Authentication and authorization

OIDC token validation, Azure AD, Keycloak and fine-grained group access control

Subscriptions and quotas

Model allocation to teams and self-service API key management

Consumption limiting

Token rate limiting and automated HTTP 429 rejections

Usage tracking

Traceability, consumption metrics and internal chargeback models

Local / external models

On-premise inference engines (vLLM, llm-d) and Cloud APIs (such as OpenAI, Anthropic, AWS Bedrock, Google Vertex, Azure OpenAI)

MaaS: the shared cafeteria counter

Find a model

See which approved endpoints are available

Get access

Use the team's credentials and permissions

Track usage

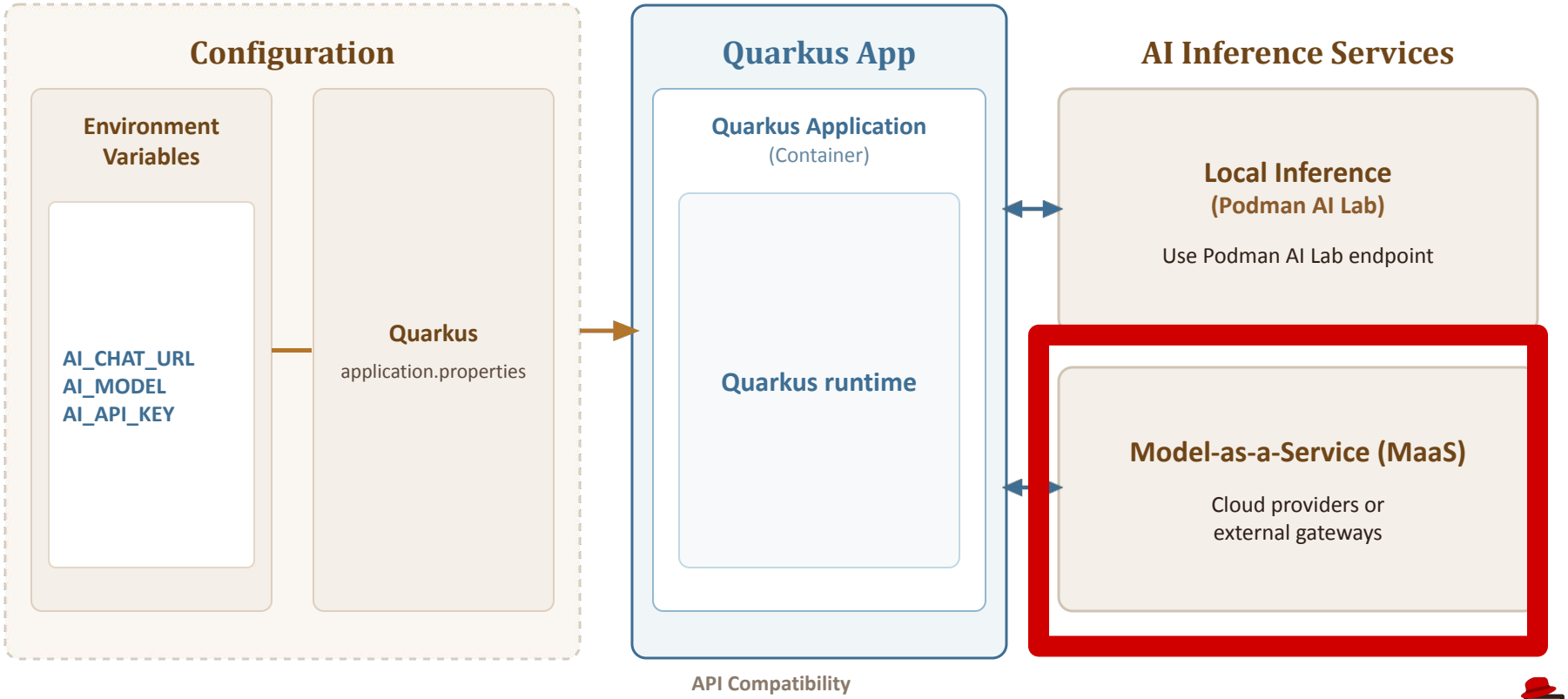
See consumption and remaining quota

Get support

Know who handles updates and incidents



Quarkus to AI Inference Architecture



Demo 03: the shared barista

OpenShift AI MaaS

A shared endpoint. Same Java checks.



Quarkus · Gradle · Java 25
React · Vite · Tailwind

GitHub: https://github.com/maarten-vandeperre/owning-the-inference-layer

The platform serves. Quarkus validates.

Coffee app

Same Java rules
Same React UI

→

MaaS endpoint

Access, quota and routing

→

OpenShift AI

Models, GPUs
and runtime operations

App teams consume the API. Platform teams run inference.



Three demos. No UI or validation changes.

Demo	Inference location	Configuration
01	OpenAI	API key and model
02	Podman AI Lab	Local service URL and model
03	OpenShift AI MaaS	Full endpoint, model and credential

Only inference configuration changes: endpoint, model and credential.



How much of the stack do we operate?

Hosted API

Call the API

Provider operates the model service

OpenAI demo

Managed platform

Deploy through the provider

The provider runs the underlying stack

Controls vary by service

Self-managed

Run the machine

Our team operates the serving stack

Podman AI Lab demo

With OpenShift AI, the platform team runs the service and app teams use it.

Platform engineering 101 (Devoxx Belgium): https://youtu.be/DaYNwrvJGmk?si=22ZzXD8rxgh4w8BD



Challenge: A new model can make the wrong coffee



Same API shape

Non-deterministic answers

Same acceptance tests

Portability: can we change the supplier?



A replaceable inference client

Endpoint and model in config

A switch we have tested

Who looks after which part?

Application team	Java validation, pricing, confirmation and evaluation
Platform team	Endpoints, GPUs, runtime tuning and operations
Red Hat AI Inference	Supported tools for serving models
OpenShift AI and OpenShift	The platform for models and applications
Open ecosystem	Projects such as vLLM and llm-d



What would rule an option out?

Quality and data

Does it work, and may we send it the data?

Cost

What does a useful result actually cost?

Performance

Is it fast enough when demand peaks?

Operations and portability

Who runs it, and can we change the provider?



Key takeaway: A bit of both

Make deliberate choice(s)



One gateway

Services we allow

Someone responsible

Make sure you have monitoring in place!!!

Where Red Hat fits

The products behind the coffee analogy

In our coffee shop	In the AI stack	Red Hat
The beans	Open models, validated and compressed	Red Hat AI models on Hugging Face
The machine	The inference server	Red Hat AI Inference Server (vLLM)
The floor manager	Serving models across GPUs	llm-d, in Red Hat AI
The counter	Model as a Service and the AI gateway	OpenShift AI · Connectivity Link
The kitchen	Shared platform, GPUs and GitOps	OpenShift · RHEL · Ansible
The tasting	Quality checks and guardrails	TrustyAI and NeMo guardrails in OpenShift AI
The home machine	Local development	Podman Desktop with AI Lab

Red Hat contributes to vLLM and llm-d alongside the wider community.

We add validation, security updates and lifecycle support, with someone to call when things go wrong.

Production is not a playdate



WHAT TO TAKE HOME

Keep the rules in Java.
Keep the model replaceable.
Keep your options open

Validate every response before acting.

Same UI. Same validation. Hosted, local or shared inference.

Today

Install Podman AI Lab.
Try an open-weight model.

Next week

Externalise the LLM config.
Test a second endpoint.

Learn more

developers.redhat.com
Tutorials and sandboxes

Scan to rate this talk

Two lines of comment are worth more to us than the score.
(But a high score is appreciated 😁





Thank you
Questions?

Camille Nigon

Specialist Solutions Architect · Red Hat

linkedin.com/in/camille-nigon

Maarten Vandeperre

Sr. Specialist Solutions Architect (AppDev & AI) · Red Hat

Advocate digital sovereignty · cloud-native development · platform engineering

maarten-vandeperre.be · linkedin.com/in/maarten-vandeperre

vllm.ai · llm-d.ai · podman-desktop.io · developers.redhat.com

Scan to rate this talk

Two lines of comment are worth more to us than the score.
(But a high score is appreciated 🤭

Reference: optimisation controls

Memory use

Quantisation and KV-cache capacity

Throughput

Continuous batching and concurrency

Shared context

Prefix caching where supported

Placement

Parallelism, scheduling and locality



Reference: the Red Hat AI model repository

Model choice

Multiple architectures and licences

Test results

Check the conditions behind the evaluations

Optimisation

Compressed model variants and tooling

Our own tests

Run the application's acceptance tests again



Reference: managed services

Option	Typical appeal	What still needs checking
Shared API	Fast experiments, variable usage	Data terms, quotas, model lifecycle
Managed capacity	Reserved throughput, less infrastructure work	Isolation, commitments, supported tuning
Self managed	Placement and lifecycle control	Capacity, utilisation, skills and support



Reference: running the service

Updates

Version the model, prompts and runtime. Keep a rollback plan.

Monitoring

Request quality, latency, errors and capacity

Access and data

Identity, secrets, quotas and data policy

Ongoing support

On-call cover, recovery and cost reviews



Reference: WeAreDevelopers material

Source pages	Covered in this narrative
2–6	Why own; hosted, managed and self-managed options
7–10, 12, 15	Model choice, architecture layers and scale pressure
11, 16–17	vLLM and llm-d
13–14	Optimisation and the Red Hat AI model repository
18–20	Performance ownership, quality, latency and workload SLOs
21–23	Decision drivers, eligible options and architecture evolution



Reference: sources and credits

WeAreDevelopers source deck · Taylor Jordan Smith

Merged Devoxx deck · supplied by the presenters

podman-desktop.io/docs/ai-lab/start-inference-server

docs.redhat.com · OpenShift AI 3.4 · Models-as-a-Service

docs.vllm.ai · llm-d.ai · huggingface.co/RedHatAI

